



KodeKloud

© Copyright KodeKloud

Follow us on <https://kodekloud.com/> to learn more.

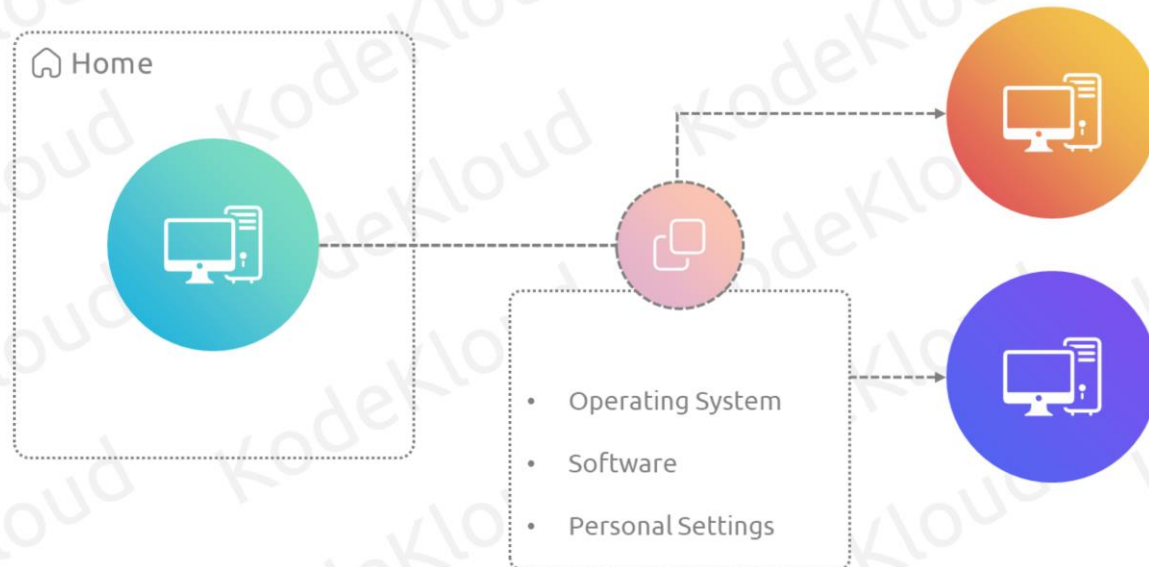


AMI and Why It is Needed

© Copyright KodeKloud

In this lecture we will learn about AMI's and the need of it.

Amazon Machine Images (AMIs)



© Copyright KodeKloud

Imagine you have a computer at home, and you want to share a fully set up and customized computer with your friends.

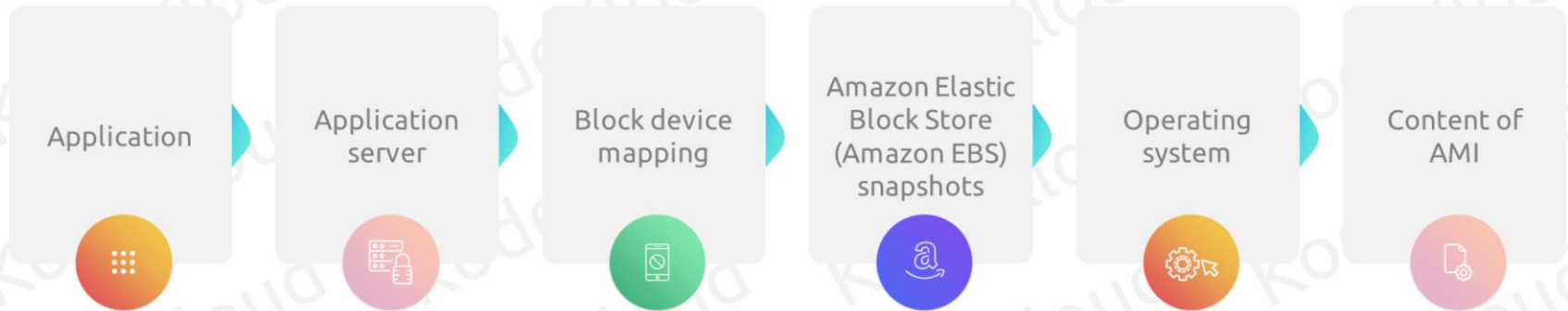
Instead of giving them your physical computer, you create a clone or image of your computer's hard drive. This image contains everything on your computer, including the operating system, software, and personal settings.

You provide this clone to your friends, who can then use it to create exact copies of your computer on their own

machines.

They don't need to go through the hassle of setting up the operating system, installing software, or configuring settings. They can simply use the clone to create an identical copy of your computer, including all your customizations.

Amazon Machine Images (AMIs)



© Copyright KodeKloud

In AWS, an AMI works in a similar way. It's like a snapshot or clone of a virtual server in the cloud. It captures everything on that server, including the operating system, software, and any data or configurations. This AMI can be saved and reused to create new EC2 instances in AWS.

Contents of AMI are:

- Application - your application program which is running on EC2.
- It can also consist of application server.
- block device mapping - device mapping that specifies the volumes to attach to the instance when it's launched.
- EBS snapshot – Elastic block store snapshot or we can relate it to disk snapshot
- Operating system - different AMI's are available linux , Windows and macos

These are the contents of AMI.

- AMI can be obtained for free and paid from Amazon marketplace. We will look at this when we launch an EC2 instance later in the course

Amazon Machine Images (AMIs) – Advantages

01



Easy
replication

02



Faster
deployment

03



Consistency and
standardization

04



Scalability

05



Versioning and
rollback

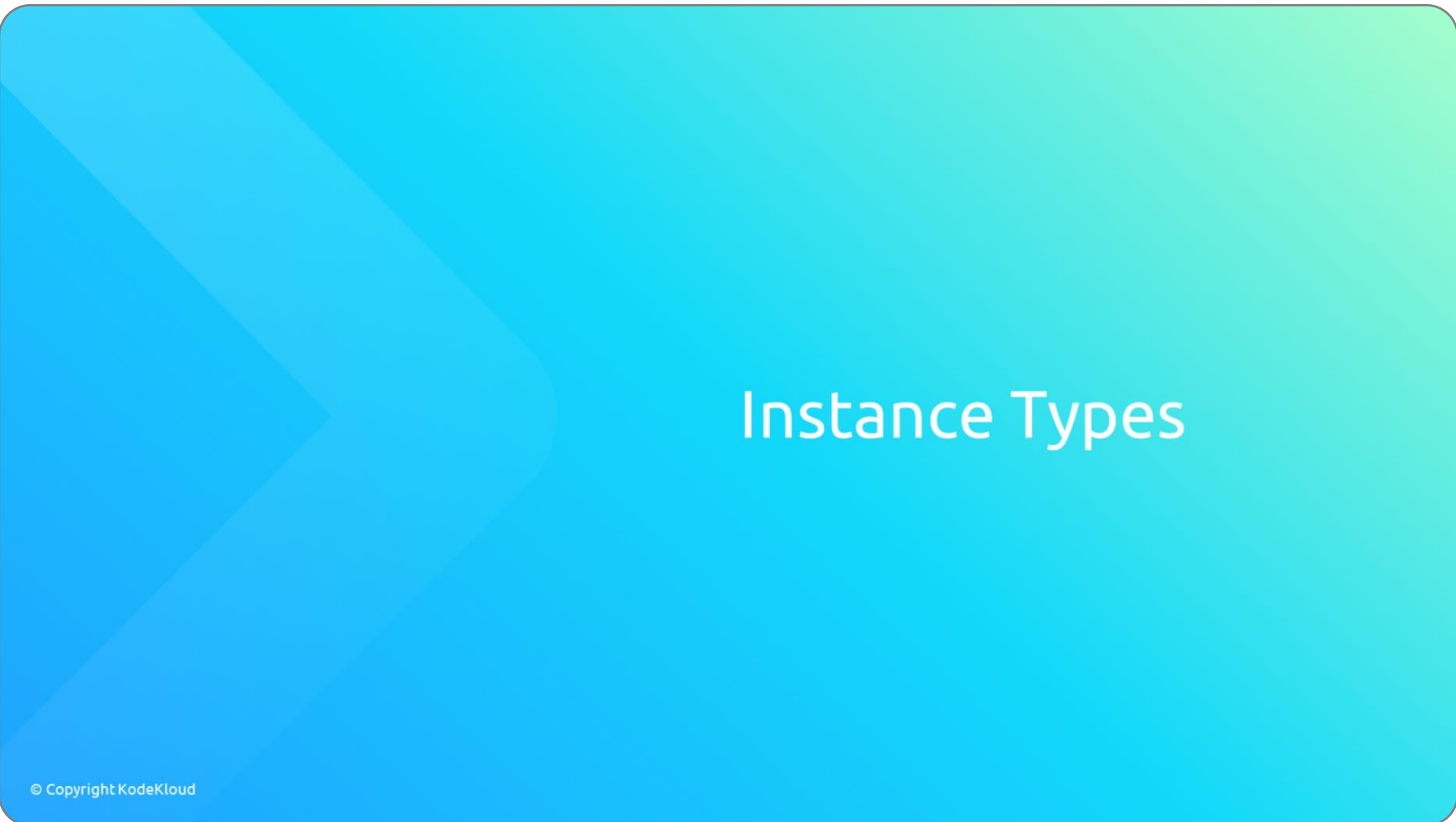
Easy Replication: AMIs allow you to easily replicate virtual servers in AWS. Instead of manually setting up each server from scratch, you can use an AMI to create identical instances quickly and efficiently

Faster Deployment: By using an AMI, you can launch new instances with pre-configured operating systems and software configurations. This saves time and effort, as you don't have to install and configure everything manually.

Consistency and Standardization: AMIs ensure consistency in your server configurations. When you use the same AMI to create multiple instances, you can be confident that they will all have the same software stack and settings, reducing the chances of errors or inconsistencies.

Scalability: AMIs are instrumental in scaling your infrastructure. You can create an AMI of a properly configured and optimized instance, and then use it to launch multiple instances that can handle increased traffic or workload demands.

Versioning and Rollbacks: AMIs support versioning, allowing you to manage updates and changes over time. This enables you to roll back to previous versions if needed or maintain different versions for different purposes.



Instance Types

© Copyright KodeKloud

In this lecture we will learn about EC2 Instance Types

Instance Types



8 GB RAM
Intel core i7



16 GB RAM
Intel core i9

Instance Type	Identifier
General Purpose	M, T
Compute Optimized	C
Memory Optimized	R, Z
Accelerated Computing	P, DL, INF, G, F, VT
Storage Optimized	I, D, H
HPC Optimized	HPC

© Copyright KodeKloud

When we want a laptop, we chose the configuration based on our use case like gaming , video editing etc. Similarly, AWS provides us multiple instance type for different purpose which will benefit users:

AWS EC2 instance types start with a letter to categorize them based on their general purpose or specialized use. Each letter signifies the primary purpose of the instance type:

- 1.General Purpose (e.g., T, M, A):** Balanced between compute, memory, and networking resources.
- 2.Compute Optimized (e.g., C):** High-performance processors, suited for compute-intensive applications.
- 3.Memory Optimized (e.g., R, X, U):** Larger memory sizes, ideal for memory-intensive applications.
- 4.Accelerated Computing (e.g., P, G, F):** Includes hardware accelerators, or co-processors, such as GPU for graphics and data processing.
- 5.Storage Optimized (e.g., I, D, H):** Designed for workloads requiring high, sequential read and write access to large datasets on local storage.
- 6. HPC :** These instances offer the high performance, fast networking, and high throughput necessary for HPC workloads.

Benefits

01



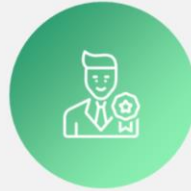
Performance
optimization

02



Cost
optimization

03



Specialized
workloads

04



Scalability and
flexibility

05



Architecture
optimization

Performance Optimization: Different workloads have varying resource demands. By offering various instance types, AWS allows you to select the most suitable instance type that optimizes performance for your specific workload. You can choose an instance type with the right balance of CPU, memory, storage, and network capacity to achieve optimal performance.

Cost Optimization: Different instance types have different pricing structures. AWS provides instance types with various

performance levels and pricing options, allowing you to select the most cost-effective option for your workload. You can choose an instance type that aligns with your performance requirements while optimizing costs.

Specialized Workloads: Certain workloads require specialized resources, such as high-memory capacity, powerful GPUs for graphics processing, or FPGA for hardware acceleration. AWS offers specific instance types tailored to these specialized workloads, enabling you to achieve the best performance and efficiency for your specific use case.

Scalability and Flexibility: With a wide range of instance types, AWS allows you to scale your infrastructure based on the evolving needs of your workload. You can easily resize or switch to a different instance type to accommodate changing demands without the need for significant architectural changes.

Architecture Optimization: Different instance types have varying capabilities and features that align with specific architectural considerations. For example, compute-optimized instances are suitable for CPU-intensive applications, memory-optimized instances for data-intensive workloads, and storage-optimized instances for applications requiring high-capacity storage.

Instance Size

Instance Size	vCPU	Memory	Network Bandwidth (Gbps)	EBS Bandwidth (Gbps)
Medium	1	4	Up to 12.5	Up to 10
Large	2	4	Up to 12.5	Up to 10
Xlarge	4	8	Up to 12.5	Up to 10
2xlarge	8	16	Up to 15	Up to 10
4xlarge	16	32	Up to 15	Up to 10
8xlarge	32	64	15	10
12xlarge	48	96	22.5	15
16xlarge	64	128	30	20

© Copyright KodeKloud

Now we know different type of instance family, next we will look at different instance size which is provided generally by AWS.

AWS instance size options are , based on the instance type, ie is memory, compute, the size might vary

- Medium
- Large

- Xlarge
- 2xlarge
- 4xlarge
- 8xlarge
- 12xlarge
- 16xlarge

Now let's look at the specification of these instance size, these are general instance type which are commonly used, and this is not an exhaustive list. Always refer to AWS website for latest configuration as AWS keep updating and introducing new instance type change configurations as well.

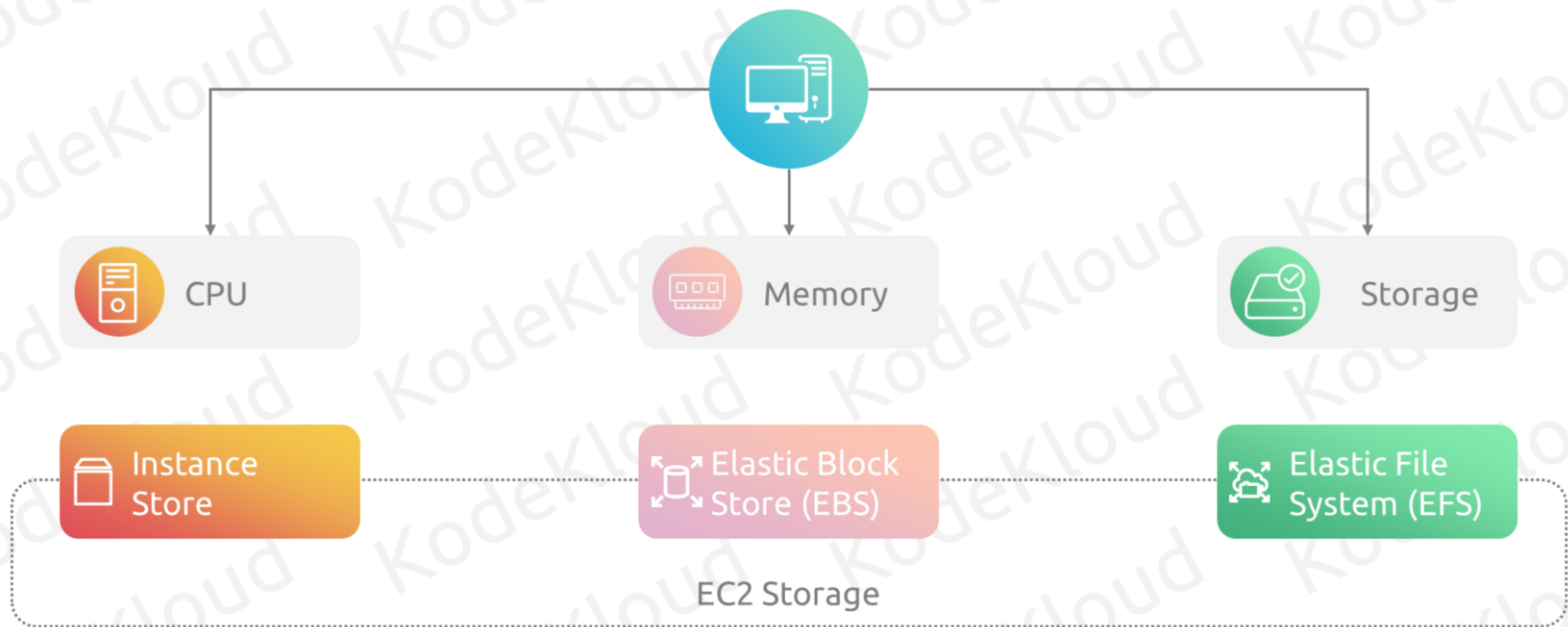
Network bandwidth and EBS bandwidth will vary based on the instance type, hence it is important to review the unitization always.

EC2 Storage

© Copyright KodeKloud

In this lecture we will learn about EC2 storage

EC2 Storage



© Copyright KodeKloud

A computer / VM can be broadly classified into compute (which is RAM & CPU) and storage, where we store file like such as documents, photos, and videos.

Amazon EC2 provides you with flexible, cost effective, and easy-to-use data storage options for your instances. Each option has a unique combination of performance and durability. These storage options can be used independently or in

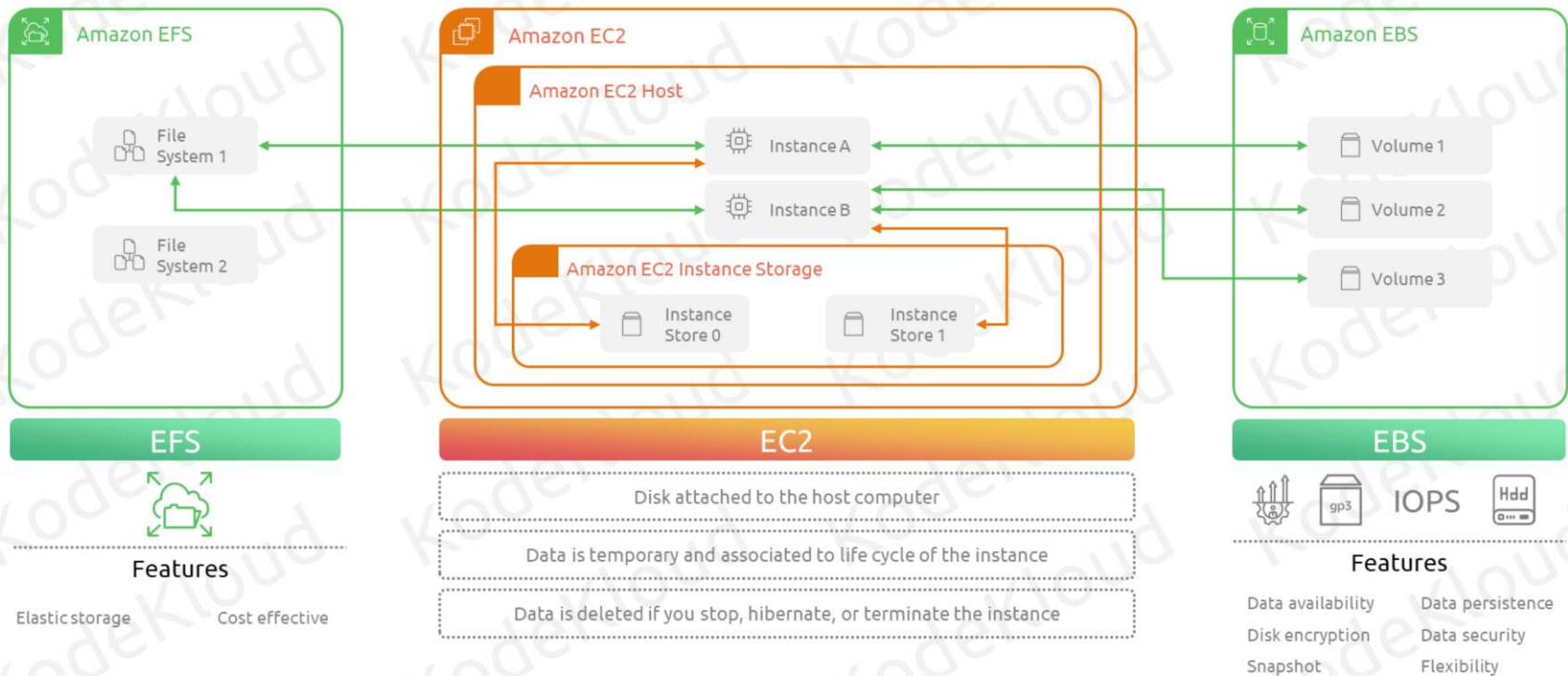
combination to suit your requirements.

Here are the commonly available storage options in EC2 are:

1. Instance Store:
2. Amazon Elastic Block Store (EBS):
3. EFS Elastic File System

We will look into these options in details next

EC2 Storage



© Copyright KodeKloud

It is important to understand the relationship of these storage type with EC2 instance, features of this storage and when should it be used.

Instance store:

- Some EC2 instance types come with temporary storage known as instance store.

- Instance store volumes provide high-performance, low-latency storage that is physically attached to the EC2 host server.
- Unlike EBS, instance store volumes are not persistent and are lost if the EC2 instance is stopped, terminated, or fails.
- Instance store is suitable for temporary data, caching, and scratch space where data persistence is not a requirement.
- Apart from reboot, initiated by user or system events the content in the instance is deleted for all other events like stop, hibernate etc,

Elastic File System

Amazon EFS provides scalable file storage for use with Amazon EC2. You can use an EFS file system as a common data source for workloads and applications running on multiple instances.

Features of EFS are

Elastic Storage

Unlike EBS, where the size is provisioned in advance irrespective of whether we use it or not, in EFS storage can increase and reduce dynamically based on the usage.

Cost Effective

Pay only for the storage you use,

Amazon Elastic Block Store (EBS):

- EBS provides block-level storage volumes that can be attached to EC2 instances. (show animation when the EBS volumes are attached to the instance)
- EBS volumes are highly durable and persistent, meaning they retain data even if the EC2 instance is stopped or terminated.

- EBS volumes offer different types optimized for different workloads, such as
 - General Purpose SSD ,
General Purpose SSD volumes (gp2 and gp3) balance price and performance for a wide variety of transactional workloads. These volumes are ideal for use cases such as boot volumes, medium-size single instance databases, and development and test environments.
 - * Provisioned IOPS SSD ,
Provisioned IOPS SSD volumes (io1 and io2) are designed to meet the needs of I/O-intensive workloads that are sensitive to storage performance and consistency. They provide a consistent IOPS rate that you specify when you create the volume. This enables you to predictably scale to tens of thousands of IOPS per instance
 - Throughput Optimized HDD (st1).
Throughput Optimized HDD volumes (st1) provide low-cost magnetic storage that defines performance in terms of throughput rather than IOPS. These volumes are ideal for large, sequential workloads such as Amazon EMR, ETL, data warehouses, and log processing.

Features of EBS

Data availability

When you create an EBS volume, it is automatically replicated within its Availability Zone to prevent data loss due to failure of any single hardware component. You can attach an EBS volume to any EC2 instance in the same Availability Zone. After you attach a volume, it appears as a native block device similar to a hard drive or other physical device. At that point, the instance can interact with the volume just as it would with a local drive. You can connect to the instance and format the EBS volume with a file system, such as ext3, and then install applications.

Data persistence

An EBS volume is off-instance storage that can persist independently from the life of an instance. You continue to pay for the volume usage as long as the data persists.

Disk Encryption

You can use encrypted EBS volumes to meet a wide range of data-at-rest encryption requirements for regulated/audited data and applications. Amazon EBS encryption uses 256-bit Advanced Encryption Standard algorithms (AES-256) and an Amazon-managed key infrastructure.

Data Security

Amazon EBS volumes are presented to you as raw, unformatted block devices, (that is, the raw blocks are zeroed) prior to any use or re-use by a customer.

Snapshot

Amazon EBS provides the ability to create snapshots (backups) of any EBS volume and write a copy of the data in the volume to Amazon S3,

Flexibility

You can modify volume type, volume size, and IOPS capacity without service interruptions



Create EC2 Instance Using CLI

© Copyright KodeKloud

In this lecture we will learn how to create EC2 instance using CLI

AWS CLI

```
aws EC2 help
```

```
aws ec2 run-instances
```



Description



Available Commands



Options



Command to run EC2 Instance



Description



Options



Examples

Mandatory Inputs for EC2 Instance

```
aws ec2 run-instances \  
  
--image-id ami-05f2f979afcf322a \  
  
--instance-type t2.micro
```



Image ID



Instance Type

Image id and instance type are two mandatory inputs to create an EC2 Instance

AWS CLI - Find AMI images

```
aws ec2 describe-images --owners self amazon
```

```
"Images": [  
  {  
    "Architecture": "x86_64",  
    "CreationDate": "2023-04-04T18:10:12.000Z",  
    "ImageId": "ami-056b35f582f6afbe7",  
    "ImageLocation": "amazon/Cloud9AmazonLinux2-2023-04-04T17-37",  
    "ImageType": "machine",  
    "Public": true,  
    "OwnerId": "140476768419",  
    "PlatformDetails": "Linux/UNIX",  
    "UsageOperation": "RunInstances",  
    "State": "available",  
    "BlockDeviceMappings": [  
      {
```

```
        "DeviceName": "/dev/xvda",  
        "Ebs": {  
          "DeleteOnTermination": true,  
          "SnapshotId": "snap-0875cc8a125bc63fd",  
          "VolumeSize": 10,  
          "VolumeType": "gp2",  
          "Encrypted": false  
        }  
      }  
    ]
```

© Copyright KodeKloud

AWS EC2 describe-images , This command would return a list of AMIs that you have permission to launch and the ones provided by Amazon

Terminal

```
aws ec2 run-instances --image-id ami-056b35f582f6afbe7 --instance-type t2.micro
```

```
{
  "Groups": [],
  "Instances": [
    {
      "AmiLaunchIndex": 0,
      "ImageId": "ami-056b35f582f6afbe7",
      "InstanceId": "i-076290d4a72165019",
      "InstanceType": "t2.micro",
      "LaunchTime": "2023-09-13T06:35:11+00:00",
      "Monitoring": {
        "State": "disabled"
      },
      "Placement": {
        "AvailabilityZone": "ap-southeast-1a",
        "GroupName": "",
        "Tenancy": "default"
      },

```

Terminal

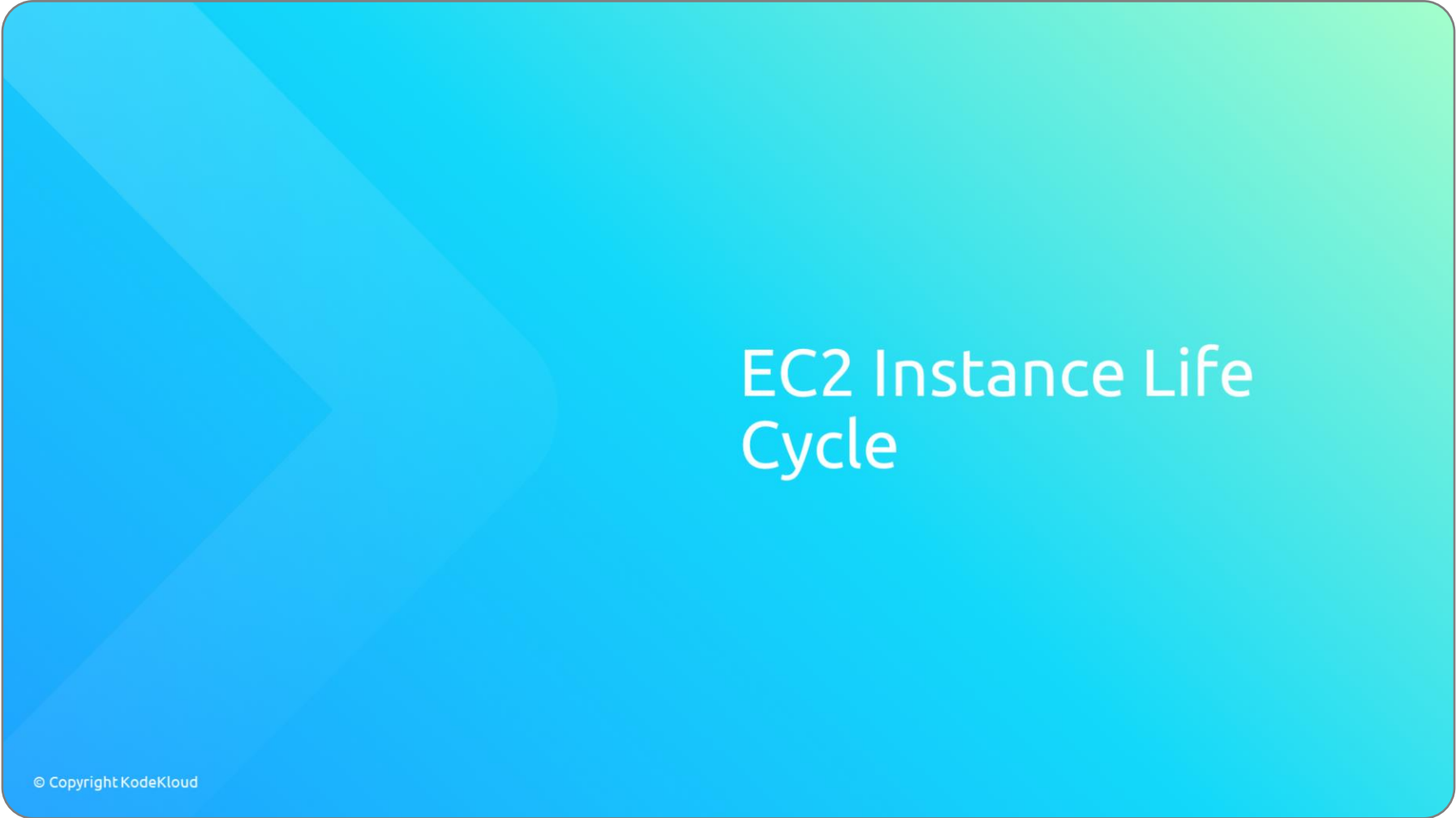
```
    "PrivateDnsName": "ip-172-31-39-161.ap-southeast-1.compute.internal",
    "PrivateIpAddress": "172.31.39.161",
    "ProductCodes": [],
    "PublicDnsName": "",
    "State": {
      "Code": 0,
      "Name": "pending"
    },

```

© Copyright KodeKloud

Let's create an EC2 instance from CLI, by providing the least amount of information. I.e image id and instance type

In the output you can verify the instance image ID, instance type and the private DNS Name

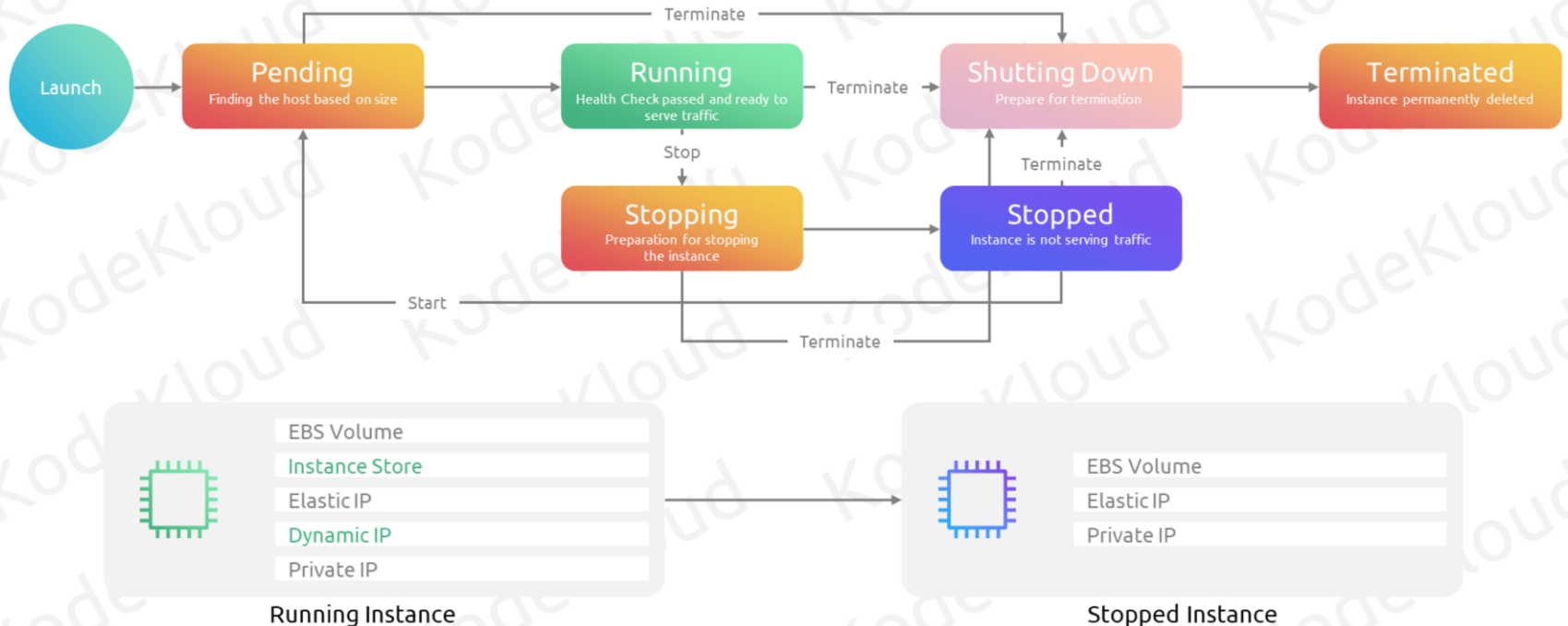
The slide features a blue-to-green gradient background. On the left side, there is a large, stylized white arrow pointing to the right. The text "EC2 Instance Life Cycle" is displayed in white on the right side of the slide.

EC2 Instance Life Cycle

© Copyright KodeKloud

In this lecture we will learn about EC2 instance Life Cycle

EC2 Instance Life Cycle



© Copyright KodeKloud

An Amazon EC2 instance transitions through different states, from the moment you launch it through to its termination.

Pending

When launch the EC2 instance, first state is pending, while in this state AWS is trying to find a host to launch the EC2 instance based on the instance type which was selected while creating.

Running

After pending state, a health check of the EC2 instance is performed by AWS, once the health check has passed, instance will move to running state.

- * During the running state data in the instance store is available.
- * Based on the VPC settings Dynamic IP is assigned to the EC2 instance automatically.

Stopping : When a stop signal is passed to EC2 instance , by AWS console, CLI or even the OS command, the EC2 instance enters stopping state.

Stopped: Instance has stopped running and not serving any traffic, but only the resources like EBS volume, Elastic IP and private IP are still retained.

Shutting-Down

When we terminate a stopped instance, The instance state will show as shutting-down, the is being prepared for termination.

Terminated

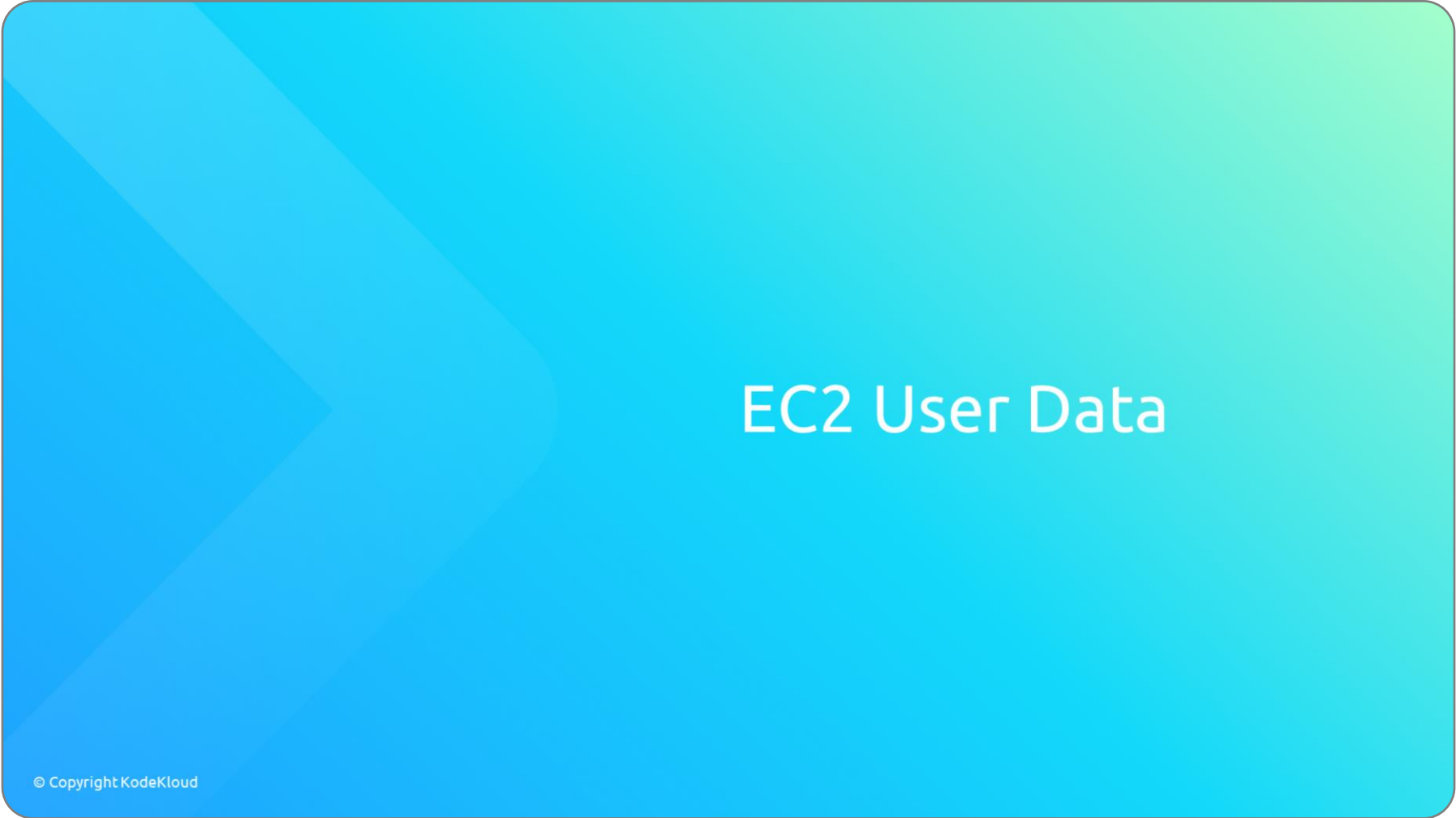
The instance has been permanently removed from EC2 service, compute resources like CPU/RAM are released for reuse.

Edge Cases:

Start/ Termination : When the instance is in stopping or stopped state, terminate stage can be triggered which will change the status to shutting-down, this is happening at the backend.

Terminate

If health check fails during pending stage, the instance state is changed to shutting-down by passing terminate signal automatically

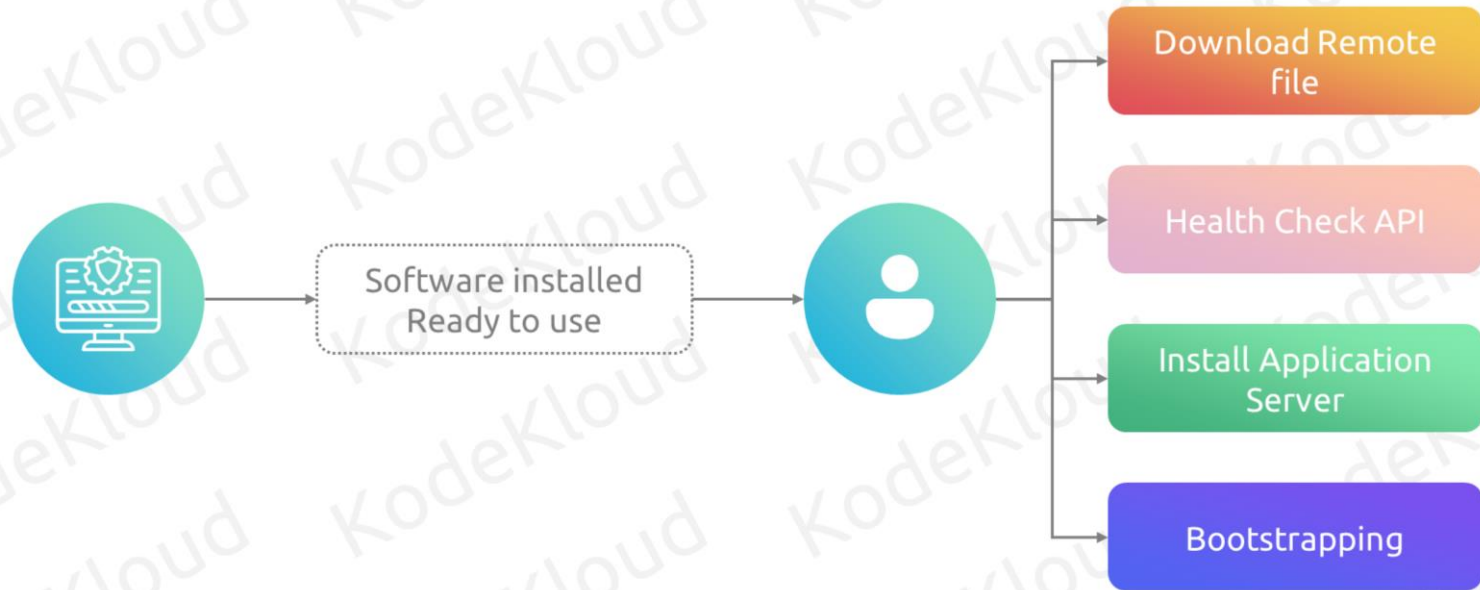
The slide features a vibrant blue-to-green gradient background. On the left side, there is a large, stylized white arrow pointing towards the right. The text 'EC2 User Data' is centered on the right side of the slide in a white, sans-serif font.

EC2 User Data

© Copyright KodeKloud

In this lecture we will learn about EC2 User Data

EC2 User Data



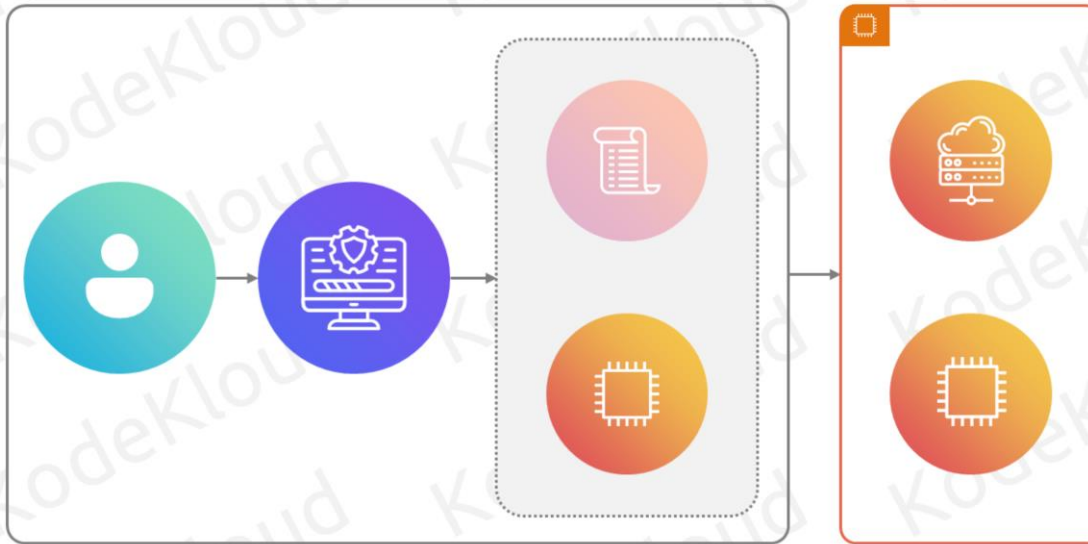
© Copyright KodeKloud

if you have use cases like when an EC2 instances is created, you want to perform few tasks automatically like

1. Downloading remote file
2. Validating health check API
3. Install packages
4. Bootstrapping your ec2 instance to be ready in the target state

We can use EC2 User data for all these actions.

EC2 User Data



Executed only on instance launch, not during reboot



User data must be base64-encoded

16 KB

User data is limited to 16 KB



User data is treated as opaque data



Longer script execution can increase boot-up time

User provides the user data script while creating the instance, EC2 instance will execute the script provided.

This feature is used for pre-configuring/bootstraping the instance with require configuration before the EC2 instance is ready to server traffic.

Few things to be aware of

- If you stop an instance, modify its user data, and start the instance, the updated user data is not run when you start the instance.
- User data must be base64-encoded. The Amazon EC2 console can perform the base64-encoding for you or accept base64-encoded input.
- User data is limited to 16 KB, in raw form, before it is base64-encoded.
- * User data must be base64-decoded when you retrieve it. If you retrieve the data using instance metadata or the console, it's decoded for you automatically.
- User data is treated as opaque data: what you give is what you get back. It is up to the instance to be able to interpret it.
- Longer user data script can impact system boot time



KodeKloud

© Copyright KodeKloud

Follow us on <https://kodekloud.com/> to learn more.